

CSE 440: NLP

Midterm Prep from books

Turjoy Das

March 29, 2026

Contents

1	Chapter 4: Naive Bayes, Text Classification, and Sentiment Analysis	4
1.1	3.5.1 N-grams and their Application	4
1.2	4.1 Naive Bayes Classifiers	4
1.3	4.2 Training the Naive Bayes Classifier	5
1.4	4.3 Working Example of Naive Bayes	5
1.5	4.4 Optimizing for Sentiment Analysis	6
1.6	4.5 Naive Bayes as a Language Model	6
1.7	4.7 Evaluation: Precision, Recall, F-measure	6
1.8	4.10 Avoiding Harms in Classification	7
2	Chapter 5: Logistic Regression	8
2.1	5.1 The Sigmoid Function	8
2.2	5.2 Classification with Logistic Regression	8
2.3	5.2.1 Sentiment Classification	9
2.4	5.4 Learning in Logistic Regression	9
2.5	5.5 The Cross-Entropy Loss Function	10
2.6	5.7 Regularization	11
3	Chapter 6: Vector Semantics and Embeddings	12
3.1	6.1 Lexical Semantics	12
3.2	6.2 Vector Semantics	12
3.3	6.3 Words and Vectors	13
3.4	6.4 Cosine for Measuring Similarity	14
3.5	6.5 TF-IDF: Weighing Terms in the Vector	14
3.6	6.6 Pointwise Mutual Information (PMI)	15
3.7	6.7 Applications of the TF-IDF or PMI Vector Models	16

4	Chapter 7: Neural Networks and Neural Language Models	17
4.1	Core Concepts	17
4.2	Feedforward Neural Networks	18
4.3	Backpropagation and Training	20
4.4	Deep Learning and Neural Language Models	21
4.5	Neural Language Models	22
5	Question set 1:	23
5.1	Question 1:	23
5.1.1	Part A: How can overfitting be detected in a machine learning model? What are the general techniques you can use to reduce overfitting? .	23
5.1.2	Part B: You have built a spam detector model, and that model was tested on 100 emails. Out of these 100 emails, 80 were spams and 20 were not spams. Your model identified 90 of them as spam and 10 of them not spams. All the not spams the model identified are correct. Now, build a confusion matrix and calculate precision, recall, and F-score for the spam class.	24
5.2	Question set 2:	25
5.2.1	Part A: Suppose you have a collection of 3 documents and a term "AI" appears in them as follows:	25
5.2.2	Part B: With appropriate examples, write short notes on (what it is, what are the challenges, how to solve them etc.):	25
5.3	Question 3:	26
5.3.1	Part A: The Book presents three semantic properties of word embed- dings: Different types of Similarity or Association, Analogy/Relational Similarity, and Historical Semantics. Write short notes on each of them with appropriate examples.	26
5.3.2	Part B: Compare binary cross-entropy of two models	27
6	Question set 2:	28
6.1	Question 1:	28
6.1.1	Part A: You are training a deep neural network, and the training ac- curacy is 98%, but the validation accuracy is only 70%. What does this indicate, and how can you address it?	28
6.1.2	Part B: What is the impact of increasing the number of features in a dataset on underfitting and overfitting? Explain.	29
6.1.3	Part C: Is overfitting always bad? Discuss one scenario where a slightly overfit model might be acceptable.	29
6.2	Question 2:	30
6.2.1	Part A: Average binary cross-entropy calculation	30
6.2.2	Part B: Describe:	30
6.3	Question 3:	31
6.3.1	Part A: A company is analyzing customer reviews using NLP tech- niques. Consider the following matrix for three documents and three terms:	31

6.3.2	Part B: Explain how the TF-IDF values would change if the term "price" appeared in every document.	33
7	Question Set 3:	33
7.1	Question 1:	33
7.1.1	Part A: Explain with examples:	33
7.1.2	Part B: Compare two classifiers using cross-entropy	34
7.2	Question 2:	35
7.2.1	Part A: Suppose you have a collection of 3 documents and a term "AI" appears in them as follows:	35
7.2.2	Part B: Explain how the TF-IDF values would change if the term "price" appeared in every document.	36
7.3	Question 3:	36
7.3.1	Part A: Gender bias in word embeddings means some words are closer to male words and other words are closer to female words. For example, a poorly trained word vector may associate 'doctor' to 'man' and 'nurse' to 'woman'. If I tell you your embedding has this issue, how will you test it? How can you solve it? Explain.	36
7.3.2	Part B: Explain the training and testing process of the Naive Bayes algorithm for sentiment classification. That is, you are given N sentences where some are positive sentiment sentences and some are negative, and you have to train a Naive Bayes model that will be able to classify a sentence. Then, you got a new sentence which you do not know whether it is positive and negative, and you need to figure out. How are you going to do it?	36

1 Chapter 4: Naive Bayes, Text Classification, and Sentiment Analysis

1.1 3.5.1 N-grams and their Application

Q1. What is an N-gram?

An N-gram is a contiguous sequence of N items from a given sample of text or speech. In text classification, N-grams are used to represent sequences of words or characters, helping to capture contextual information in the data. N-grams allow the model to learn both word-level and phrase-level relationships.

- A **1-gram** (unigram) represents individual words. For example, in the sentence "I love machine learning," the unigrams are: "I", "love", "machine", "learning". - A **2-gram** (bigram) represents pairs of consecutive words. For example, from the same sentence, the bigrams are: "I love", "love machine", "machine learning". - A **3-gram** (trigram) represents triplets of consecutive words. For example, "I love machine", "love machine learning" would be the trigrams.

Q2. How are N-grams used in text classification?

N-grams are particularly useful in text classification tasks because they capture the relationships between words based on their proximity to each other. For instance, in sentiment analysis, a unigram may represent individual word occurrences, while bigrams and trigrams can capture contextual word pairs or triples that indicate sentiment more accurately. For example, a unigram like "good" may indicate a positive sentiment, but the bigram "not good" could indicate a negative sentiment.

1.2 4.1 Naive Bayes Classifiers

Q3. What is a Naive Bayes classifier?

The Naive Bayes classifier is a probabilistic model based on **Bayes' theorem**. It is called "naive" because it assumes that the features (input variables) are conditionally independent, given the class label. This strong assumption simplifies the computation and makes Naive Bayes a highly efficient algorithm for classification tasks like spam detection, sentiment analysis, and document classification.

Bayes' theorem is given by:

$$P(c|x) = \frac{P(c)P(x|c)}{P(x)}$$

where: - $P(c|x)$ is the posterior probability of class c given the features x , - $P(c)$ is the prior probability of class c , - $P(x|c)$ is the likelihood of observing features x given class c , - $P(x)$ is the evidence or normalization factor.

Q4. What are the components of the Naive Bayes model?

The Naive Bayes model consists of: - **Prior Probability** $P(c)$: The probability of a document belonging to a particular class, without considering the features. - **Likelihood** $P(x|c)$: The probability of observing the features given the class. - **Posterior Probability**

$P(c|x)$: The probability of a class given the observed features, which we want to maximize in order to make a classification decision.

Q5. How does the Naive Bayes classifier work in practice?

To classify a new document, the Naive Bayes classifier computes the posterior probability for each class based on the features present in the document and assigns the class with the highest posterior probability. This is done for each class c :

$$P(c|x) \propto P(c) \prod_{i=1}^n P(x_i|c)$$

where $x = [x_1, x_2, \dots, x_n]$ is the vector of features (e.g., words in a document).

1.3 4.2 Training the Naive Bayes Classifier

Q6. How are the probabilities $P(c)$ and $P(x|c)$ learned in Naive Bayes?

- **Prior Probability $P(c)$ ** is calculated by counting how many documents belong to class c in the training set:

$$P(c) = \frac{\text{Number of documents in class } c}{\text{Total number of documents}}$$

- **Likelihood $P(x|c)$ ** is estimated by counting the occurrences of feature x in documents of class c , and dividing by the total number of features in class c :

$$P(x_i|c) = \frac{\text{count}(x_i \text{ in class } c)}{\text{count of all features in class } c}$$

Q7. What are stop words and how do they affect Naive Bayes?

Stop words are common words like "the", "is", "in", and "of", which carry little meaning and are usually removed during preprocessing. In Naive Bayes, stop words can skew the likelihood estimates, as they appear frequently in the data without providing much information about the class. Their removal helps in improving the accuracy of the model.

1.4 4.3 Working Example of Naive Bayes

Q8. How is Naive Bayes applied to sentiment analysis?

In sentiment analysis, Naive Bayes can be used to classify text as either positive or negative based on the frequency of words in the text. The features are the individual words or n-grams present in the text, and the model computes the likelihood for each class (positive or negative). The class with the highest posterior probability is selected:

$$P(c|x_1, x_2, \dots, x_n) = \frac{P(c) \prod_{i=1}^n P(x_i|c)}{P(x_1, x_2, \dots, x_n)}$$

where x_i are the features (words), and $P(c)$ is the prior probability of the class.

1.5 4.4 Optimizing for Sentiment Analysis

Q9. What is add-one smoothing in Naive Bayes?

Add-one smoothing (also known as Laplace smoothing) is used to handle the issue of zero probabilities for unseen features (e.g., words) in the test data. It ensures that every feature has a non-zero probability by adding 1 to the count of each feature in the training set. The smoothed likelihood for a feature x_i in class c is:

$$P(x_i|c) = \frac{\text{count}(x_i \text{ in class } c) + 1}{\text{total features in class } c + \text{number of unique features}}$$

Q10. How does add-one smoothing affect the model?

Add-one smoothing ensures that no feature has zero probability, preventing the model from assigning zero probability to unseen words during classification. This makes the model more robust, especially when there is limited training data or when new features appear during testing.

1.6 4.5 Naive Bayes as a Language Model

Q11. How is Naive Bayes used as a language model?

Naive Bayes can be used as a language model for tasks such as **language identification**. Given a document, the model computes the probability that the document belongs to a particular language based on the words or features in the document. The language with the highest posterior probability is selected:

$$P(c|x) \propto P(c) \prod_{i=1}^n P(x_i|c)$$

This approach is useful for automatic language detection in documents.

1.7 4.7 Evaluation: Precision, Recall, F-measure

Q12. What are precision and recall?

- **Precision** is the proportion of true positives among all instances classified as positive:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall** is the proportion of actual positives that are correctly identified:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Q13. What is the F-measure?

The F-measure is the harmonic mean of precision and recall, providing a balanced measure of performance. It is especially useful when there is an uneven class distribution:

$$F = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Q14. Why is F-measure used instead of just precision or recall?

The F-measure is a single metric that balances both precision and recall, providing a more comprehensive evaluation of the model's performance, particularly in cases of class imbalance.

1.8 4.10 Avoiding Harms in Classification**Q15. What are the potential harms of using classifiers in text classification?**

Classifiers may produce biased or harmful results if they are trained on data that includes biases. For example, sentiment analysis models might reflect societal biases such as gender or racial stereotypes. It is important to consider fairness and ethical implications when building and deploying these models.

Q16. How can we avoid harms in classification?

We can avoid harms by ensuring diverse and representative training data, monitoring model predictions for biases, and using techniques such as ****bias correction**** or ****fairness constraints**** during training to minimize the impact of harmful biases.

—

2 Chapter 5: Logistic Regression

2.1 5.1 The Sigmoid Function

Q1. What is the sigmoid function?

The sigmoid function is used in logistic regression to map real-valued numbers to a range between 0 and 1, making it suitable for classification problems where we interpret the output as probabilities. The function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where z is a real-valued number, typically the weighted sum of input features in logistic regression. The sigmoid function transforms any real-valued input into a value between 0 and 1, which can then be interpreted as a probability for binary classification tasks.

Q2. Why do we use the sigmoid function in logistic regression?

We use the sigmoid function because it squashes the output to a probability between 0 and 1. It maps large positive values to 1 and large negative values to 0, which is ideal for binary classification tasks. The ability of the sigmoid function to model probability makes it perfect for scenarios where the model needs to decide whether an instance belongs to class 1 or class 0.

Q3. What is the derivative of the sigmoid function?

The derivative of the sigmoid function is used in optimization algorithms like gradient descent. It is:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

This derivative is crucial during the training phase, as it is used to calculate how much the weights should be updated in the direction that minimizes the loss function. This property helps adjust weights during model training by determining how sensitive the output is to changes in the input.

2.2 5.2 Classification with Logistic Regression

Q4. How does logistic regression make a classification decision?

Logistic regression computes the probability that an instance x belongs to a particular class. The probability for class 1 is given by:

$$P(y = 1|x) = \sigma(w \cdot x + b)$$

where w is the weight vector, x is the feature vector, and b is the bias term. The decision rule is simple: if $P(y = 1|x) > 0.5$, classify as 1, otherwise classify as 0. This decision rule ensures that logistic regression produces probabilistic outputs, which can be useful in classification problems where probabilities matter.

Q5. What is the decision boundary in logistic regression?

The decision boundary is the point where the model is equally likely to classify an instance

as either class 0 or class 1, i.e., where $P(y = 1|x) = 0.5$. This boundary is defined by the equation:

$$w \cdot x + b = 0$$

At this point, the model is uncertain, and both classes have equal probability. The decision boundary is a hyperplane in the feature space, and its orientation and position depend on the weights w and bias b learned by the model.

Q6. How is the output of logistic regression interpreted?

The output of logistic regression, $\hat{y} = \sigma(w \cdot x + b)$, is interpreted as the probability of the instance belonging to class 1. A higher probability indicates a stronger belief that the instance belongs to class 1, while a lower probability suggests that the instance is more likely to belong to class 0. In practice, this probabilistic output is useful for tasks that require a measure of confidence in the classification.

2.3 5.2.1 Sentiment Classification

Q7. What is sentiment classification?

Sentiment classification is the task of determining whether a given text expresses a positive or negative sentiment. It is a type of binary classification where the labels represent positive (1) or negative (0) sentiment. Sentiment classification is widely used in applications such as movie review analysis, product feedback analysis, and social media sentiment analysis.

Q8. How are features extracted for sentiment classification?

Features in sentiment classification are typically words or phrases that contribute to the sentiment of the document. For example, words like "good," "great," and "happy" could be associated with positive sentiment, while words like "bad," "terrible," and "sad" could be associated with negative sentiment. These words or phrases are represented as numerical features, often using techniques like **bag-of-words** or **TF-IDF**, which are then input into the logistic regression model.

Q9. How do we train a logistic regression model for sentiment classification?

We train the logistic regression model by adjusting the weights and bias to minimize the error in predicting sentiment. This is done by using the **cross-entropy loss function** and **gradient descent** to optimize the parameters. The loss function helps guide the optimization process by quantifying the difference between the predicted probabilities and the actual labels.

2.4 5.4 Learning in Logistic Regression

Q10. How are the parameters w and b learned in logistic regression?

The parameters w (weights) and b (bias) are learned by minimizing the **cross-entropy loss function** using an optimization technique called **gradient descent**. Gradient descent updates the parameters iteratively in the direction that reduces the error. The learning rate controls how large a step is taken in each update, and the process continues until the model converges to the optimal set of parameters.

Q11. What is the objective function in logistic regression?

The objective function in logistic regression is the **cross-entropy loss function**, which measures the difference between the true labels and the predicted probabilities. For binary classification, it is given by:

$$L(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

where $\hat{y}$ is the predicted probability and y is the actual label (either 0 or 1). The goal is to minimize this loss function during training.

Q12. What is gradient descent?

Gradient descent is an optimization algorithm used to minimize the loss function. It works by computing the gradient (partial derivatives) of the loss with respect to the model parameters, then updating the parameters in the opposite direction of the gradient. This process iterates until the parameters converge to values that minimize the loss function.

Q13. What is the role of learning rate in gradient descent?

The learning rate controls how large a step is taken during each update. A large learning rate might lead to overshooting the optimal solution, while a small learning rate may result in slow convergence. The learning rate is a hyperparameter that needs to be tuned to ensure efficient training.

—

2.5 5.5 The Cross-Entropy Loss Function

Q14. What is the cross-entropy loss function?

The cross-entropy loss function is used to measure the difference between the predicted probability distribution and the true distribution. It is defined as:

$$L(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

where $\hat{y}$ is the predicted probability of class 1, and y is the true label (either 0 or 1). Cross-entropy is widely used in classification tasks because it directly penalizes the model for making incorrect probabilistic predictions.

Q15. Why is cross-entropy used in logistic regression?

Cross-entropy is used because it directly measures how well the predicted probabilities match the true class labels. It is a natural fit for logistic regression, where the output is a probability. Cross-entropy is differentiable, which makes it suitable for optimization techniques like gradient descent.

Q16. What happens when the predicted probability is far from the actual label?

When the predicted probability is far from the actual label, the cross-entropy loss increases, indicating a poor prediction. This guides the gradient descent algorithm to adjust the weights in order to reduce the error. Larger discrepancies between predicted and true labels lead to a higher loss.

—

2.6 5.7 Regularization

Q17. What is overfitting in logistic regression?

Overfitting occurs when the model becomes too complex and fits the noise or random fluctuations in the training data rather than the underlying pattern. As a result, the model performs well on the training data but poorly on unseen test data. This is particularly common when there is insufficient training data or when the model is too flexible.

Q18. How does regularization help in logistic regression?

Regularization adds a penalty term to the loss function to discourage large weights. This helps to prevent overfitting by forcing the model to find simpler solutions that generalize better to unseen data. Regularization techniques like **L1** and **L2** penalize large weights, effectively reducing model complexity.

Q19. What is L2 regularization?

L2 regularization adds a penalty term proportional to the sum of the squares of the weights:

$$R(w) = \lambda \sum_i w_i^2$$

where λ is the regularization strength. This penalizes large weights and encourages the model to use smaller weights, leading to a more generalized model.

Q20. What is L1 regularization?

L1 regularization adds a penalty term proportional to the sum of the absolute values of the weights:

$$R(w) = \lambda \sum_i |w_i|$$

L1 regularization tends to drive some weights to zero, leading to sparse models where only a few features are selected. This makes L1 regularization useful for feature selection.

Q21. How does regularization affect the logistic regression objective function?

The regularized logistic regression objective function is:

$$\hat{w} = \arg \min_w \left[- \sum \log P(y|x; w) + \alpha R(w) \right]$$

where α is a regularization parameter that controls the trade-off between fitting the data well and keeping the model simple. Regularization helps prevent overfitting by discouraging large or irrelevant weights.

Q22. How do L1 and L2 regularization differ?

- **L1 regularization** encourages sparse models where some weights are exactly zero. - **L2 regularization** discourages large weights but does not necessarily drive them to zero, leading to a smoother model.

This **polished version** of Chapter 5 is now expanded with **detailed explanations**, **examples**, and **conceptual clarity**. If you would like further adjustments or would like to move to the next chapter, let me know!

3 Chapter 6: Vector Semantics and Embeddings

3.1 6.1 Lexical Semantics

Q1. What is lexical semantics?

Lexical semantics is the branch of linguistics that studies the meaning of words and their relationships to other words. It involves understanding how words represent objects, actions, and concepts, and how they can be grouped, related, and contrasted. Lexical semantics deals with various kinds of meaning relationships, such as synonymy (similar meanings), antonymy (opposites), and hyponymy (subcategories).

Q2. What are synonyms?

Synonyms are words that have the same or nearly the same meaning. For example, "big" and "large" are synonyms because they both convey a similar concept of size. In text classification or sentiment analysis, recognizing synonyms is important because they carry the same sentiment or meaning even though the specific word used may differ.

Q3. What is the principle of contrast?

The principle of contrast refers to the way words that have opposite meanings are understood in relation to each other. For instance, the words "hot" and "cold" are contrasting terms, where the meaning of one word (hot) is defined in opposition to the other word (cold). This principle is crucial in understanding antonyms, which are words with opposite meanings.

Q4. What is word similarity?

Word similarity refers to the degree to which two words share a similar meaning. It can be quantified using vector representations of words in a vector space model. Words with similar meanings tend to have closer vector representations in this space. For example, "dog" and "cat" are semantically similar, while "dog" and "car" are less similar.

Q5. What is a semantic frame?

A semantic frame is a set of words that together provide context for the meaning of a word. For example, the frame for the word "buying" might include words like "seller," "price," and "transaction." These related words help give a full understanding of the concept expressed by a single word, making frames a useful tool in natural language processing tasks.

Q6. What are word relations such as antonyms and meronyms?

- **Antonyms** are words with opposite meanings. For instance, "light" and "dark" are antonyms. - **Meronyms** are words that denote a part of a whole. For example, "wheel" is a meronym of "car," as a wheel is a part of a car. These relations are important for understanding the relationships between words in context.

3.2 6.2 Vector Semantics

Q7. What is vector semantics?

Vector semantics is an approach in natural language processing (NLP) that represents words and their meanings as vectors in a continuous, high-dimensional space. In this model, words that are semantically similar are represented by vectors that are close to each other, while words with different meanings are represented by vectors that are further apart. Word embeddings, such as Word2Vec, GloVe, and FastText, are examples of vector semantic models.

Q8. What is the significance of Osgood's representation of words in a 3-dimensional space?

Osgood's semantic differential theory posits that words can be represented in a three-dimensional space based on three fundamental dimensions of meaning: - **Evaluation** (good vs. bad), - **Potency** (strong vs. weak), - **Activity** (active vs. passive).

This framework laid the foundation for later models of semantic representation, although modern vector spaces tend to use higher dimensions to capture more nuanced meanings. The key idea is that word meanings can be understood in terms of these basic evaluative axes.

Q9. What is t-SNE in vector semantics?

t-SNE (t-Distributed Stochastic Neighbor Embedding) is a dimensionality reduction technique commonly used to visualize high-dimensional data, such as word embeddings. It works by mapping data points into a lower-dimensional space (2D or 3D) while preserving the local structure of the data. In the context of word embeddings, t-SNE helps visualize the relative positions of words and how semantically similar words are grouped together in the vector space.

Q10. How are words represented in vector semantics?

Words are represented as vectors in a multi-dimensional space where each dimension corresponds to a specific aspect of meaning. The position of a word in the space is determined by its usage in a large corpus of text. For instance, words like "king" and "queen" would be close to each other in vector space, while "king" and "dog" would be farther apart. These vector representations capture semantic relationships, such as similarity, analogy, and word associations, based on context.

3.3 6.3 Words and Vectors

Q11. What is the term-document matrix?

The term-document matrix is a matrix that represents the frequency of terms (words) in a set of documents. Each row represents a document, and each column corresponds to a term. The entries in the matrix indicate the frequency with which the term appears in the respective document. This matrix is a fundamental representation in information retrieval systems and text classification tasks, where it is used to identify patterns in the document corpus.

Q12. What is the difference between term-document and word-word matrices?

- **Term-document matrix**: Represents the frequency of terms in documents. Each document is represented as a vector of word frequencies. - **Word-word matrix**: Represents the co-occurrence of words in a specific context, such as within a sliding window of words in a document. The entries in this matrix capture how frequently two words appear together in a given context. Word-word matrices are often used to compute word similarity or association based on co-occurrence patterns.

Q13. What is the meaning of a word vector's dimensions?

In vector semantics, a word vector's dimensions correspond to various semantic properties of the word. Each dimension represents a different aspect of meaning learned from the context in which the word appears in the corpus. For example, in a 100-dimensional space, each dimension could represent features like gender, size, or color. These dimensions capture the

multifaceted nature of word meaning.

Q14. How do we calculate the similarity between two word vectors?

The similarity between two word vectors is typically calculated using **cosine similarity**, which measures the cosine of the angle between two vectors:

$$\text{cosine similarity}(w_1, w_2) = \frac{w_1 \cdot w_2}{\|w_1\| \|w_2\|}$$

Where w_1 and w_2 are the word vectors. The result ranges from -1 (completely dissimilar) to 1 (completely similar), with values closer to 1 indicating that the words are semantically similar.

3.4 6.4 Cosine for Measuring Similarity

Q15. What is cosine similarity?

Cosine similarity is a metric used to measure the cosine of the angle between two vectors. It ranges from -1 to 1, where: - 1 means the vectors are identical (i.e., the words are semantically similar), - 0 means the vectors are orthogonal (i.e., the words are not related), - -1 means the vectors are opposite in direction (i.e., the words have completely opposite meanings).

The formula for cosine similarity is:

$$\text{cosine similarity}(w_1, w_2) = \frac{w_1 \cdot w_2}{\|w_1\| \|w_2\|}$$

where w_1 and w_2 are the word vectors. Cosine similarity is widely used in NLP tasks such as document similarity and word similarity because it effectively captures the closeness of two words in vector space, regardless of their magnitude.

Q16. Why is cosine similarity commonly used?

Cosine similarity is particularly useful because it is independent of the length of the vectors. In many NLP tasks, the magnitude of word vectors may vary due to document length or frequency of words, but the cosine similarity focuses on the direction of the vectors. This makes it an effective similarity measure for word embeddings, as it compares the semantic content of words regardless of their frequency.

3.5 6.5 TF-IDF: Weighing Terms in the Vector

Q17. What is the TF-IDF score?

The TF-IDF (Term Frequency-Inverse Document Frequency) score is a statistical measure used to evaluate the importance of a word in a document relative to the entire corpus. It balances two components: - **Term Frequency (TF)**: Measures how frequently a word occurs in a document. It gives an indication of the importance of the word within that document. - **Inverse Document Frequency (IDF)**: Measures how important a word is across the entire corpus. Words that appear in many documents are given lower importance.

The formula for TF-IDF is:

$$\text{TF-IDF}(w) = \text{TF}(w) \times \text{IDF}(w)$$

where:

$$\text{TF}(w) = \frac{\text{Count of } w \text{ in document}}{\text{Total terms in document}}$$
$$\text{IDF}(w) = \log \left(\frac{\text{Total documents}}{\text{Number of documents containing } w} \right)$$

The TF-IDF score increases with the frequency of a term in a document and decreases with its occurrence across many documents in the corpus.

Q18. Why is TF-IDF important?

TF-IDF is important because it helps identify words that are both frequent in a particular document and rare across the entire corpus. These words are often more meaningful in capturing the content of the document. For example, in a collection of news articles, the term "election" may appear often in articles about politics but rarely in other topics, making it a strong feature for identifying relevant documents.

Q19. What is the relationship between term frequency and document frequency?

- **Term Frequency (TF)** measures the frequency of a term in a document, showing how important a word is within the specific document. - **Document Frequency (DF)** measures how many documents contain a particular term. If a word appears in many documents, its DF will be high, and its IDF will be low, indicating it is less significant in distinguishing one document from another.

The TF-IDF score balances these two aspects by giving higher importance to words that occur frequently in specific documents but infrequently across all documents in the corpus. This allows it to capture more relevant words that help differentiate documents.

3.6 6.6 Pointwise Mutual Information (PMI)

Q20. What is PMI (Pointwise Mutual Information)?

Pointwise Mutual Information (PMI) measures the association between two words by comparing their joint probability with their individual probabilities. PMI is particularly useful for identifying how likely two words are to appear together in a given context compared to their individual occurrence rates. The formula for PMI is:

$$\text{PMI}(w_1, w_2) = \log \frac{P(w_1, w_2)}{P(w_1)P(w_2)}$$

where $P(w_1, w_2)$ is the probability of both w_1 and w_2 appearing together, and $P(w_1)$ and $P(w_2)$ are the individual probabilities of w_1 and w_2 .

PMI values greater than 0 indicate that the words are more likely to appear together than by chance, while negative values suggest that the words are unlikely to appear together.

Q21. How is PMI used in vector semantics?

PMI is often used to enhance word vector representations by measuring the degree to which two words are related. High PMI values suggest a strong semantic relationship between words, which is useful for clustering similar words or creating more accurate word embeddings. In the context of word embeddings, PMI helps improve the semantic relationships captured between words by incorporating co-occurrence information.

3.7 6.7 Applications of the TF-IDF or PMI Vector Models

Q22. How are TF-IDF or PMI models applied?

TF-IDF and PMI models are widely used in several NLP tasks: - **Information Retrieval**: TF-IDF helps rank documents based on the importance of query terms, improving search engine relevance. - **Document Clustering**: Words with high TF-IDF values are used to cluster documents that share similar content. - **Word Similarity**: PMI-based models can help identify words with similar meanings, which is useful for synonym detection and analogy solving. - **Recommendation Systems**: In collaborative filtering, TF-IDF can be used to identify the most relevant products or services based on user preferences and historical data.

These models enable efficient processing and analysis of text data by highlighting the most informative features of the documents.

4 Chapter 7: Neural Networks and Neural Language Models

4.1 Core Concepts

Q1. What is a neural network unit (neuron)?

A neural network unit, also known as a neuron, is the basic building block of a neural network. It takes inputs, processes them using weights and a bias, and then applies an activation function to produce an output. The weighted sum of the inputs is calculated as:

$$z = w \cdot x + b, \quad y = f(z)$$

Where w is the weight vector, x is the input vector, and b is the bias term. The function $f(z)$ is the activation function, which introduces non-linearity to the network, enabling it to learn complex patterns from the data.

Q2. Why is bias needed?

Bias is essential in neural networks because it allows the model to shift the decision boundary. Without a bias term, the decision boundary is forced to pass through the origin, which severely limits the model's ability to fit complex data. The bias enables the network to learn a more flexible decision boundary that can better fit the data.

Q3. What is an activation function?

An activation function is a mathematical function applied to the weighted sum of inputs to introduce non-linearity into the model. Without activation functions, the neural network would only be able to learn linear relationships. Common activation functions include:

$$y = f(w \cdot x + b)$$

The choice of activation function greatly influences the model's performance, as it allows the network to capture complex, non-linear patterns.

Q4. Common activation functions:

- **Sigmoid**:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

This function outputs values between 0 and 1, making it useful for probability-based tasks, especially in binary classification.

- **Tanh**:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Tanh outputs values between -1 and 1, making it zero-centered, which can lead to faster convergence compared to sigmoid.

- **ReLU (Rectified Linear Unit)**:

$$\text{ReLU}(x) = \max(0, x)$$

ReLU outputs the input if positive, and zero if negative. It is computationally efficient and widely used in deep networks due to its simplicity and effectiveness in combating the vanishing gradient problem.

Q5. Why is non-linearity important?

Non-linearity is crucial because it allows neural networks to learn and model complex relationships in the data. Without non-linear activation functions, a neural network with multiple layers would behave like a single linear transformation, significantly limiting its ability to solve complex problems.

Q6. What is a perceptron?

A perceptron is a simple, single-layer neural model used for binary classification. It computes a weighted sum of the inputs and applies a threshold to determine the output:

$$y = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

The perceptron can only solve linearly separable problems, meaning that it can only classify data that can be separated by a single straight line.

Q7. What is linear separability?

A dataset is considered linearly separable if there exists a straight line (or hyperplane in higher dimensions) that can perfectly separate the different classes. In other words, a linear decision boundary can distinguish between the classes without any errors.

Q8. What is the XOR problem?

The XOR (exclusive OR) problem is a classic example where the output is 1 when the inputs differ and 0 when they are the same. The XOR problem is not linearly separable, meaning that no single straight line can separate the classes. This is a challenge for simple models like the perceptron.

Q9. Why can't a perceptron solve XOR?

A perceptron can only solve linearly separable problems, and since XOR is not linearly separable, a perceptron cannot correctly classify the XOR problem. This limitation is what motivated the development of multi-layer neural networks, which can handle more complex, non-linear patterns.

Q10. How is XOR solved?

The XOR problem can be solved using a multi-layer neural network with at least one hidden layer. The hidden layer maps the input data into a higher-dimensional space, where the data becomes linearly separable, allowing the network to solve the problem.

4.2 Feedforward Neural Networks

Q11. What is a feedforward neural network?

A feedforward neural network is a type of artificial neural network where information flows in one direction: from the input layer to the output layer, without any feedback loops or cycles. This type of network is used in many machine learning tasks, including classification, regression, and function approximation.

Q12. What is MLP (Multi-Layer Perceptron)?

A Multi-Layer Perceptron (MLP) is a type of feedforward neural network with one or more hidden layers between the input and output layers. The addition of hidden layers allows

MLPs to model more complex relationships between inputs and outputs. MLPs are widely used for tasks like image classification, speech recognition, and language modeling.

Q13. Forward propagation equations:

In a neural network, forward propagation is the process of passing input data through the network to obtain the final output. The equations for forward propagation are:

$$h = \sigma(Wx + b), \quad z = Uh, \quad y = \text{softmax}(z)$$

Where: - h is the hidden layer output, - W is the weight matrix, - x is the input, - b is the bias term, - z is the output before applying softmax, - y is the final output after applying softmax.

Q14. What is softmax?

Softmax is an activation function used in the output layer of a neural network for multi-class classification problems. It converts raw scores (logits) into probabilities by normalizing the outputs such that they sum to 1. The softmax function is defined as:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Where z_i is the raw output score for class i , and the denominator is the sum of exponentials of all the output scores.

Q15. Why softmax over sigmoid?

Sigmoid is typically used for binary classification, as it outputs a probability for each class independently. However, softmax is preferred for multi-class classification because it provides a probability distribution over all classes, ensuring that the sum of probabilities is 1. This is important when there are multiple mutually exclusive classes.

Q16. What is a hidden layer?

A hidden layer is any layer between the input and output layers in a neural network. It is responsible for learning complex features and representations of the input data. Each neuron in the hidden layer performs a weighted sum of the inputs, applies an activation function, and passes the output to the next layer.

Q17. What is a fully connected layer?

A fully connected layer (also known as a dense layer) is a layer where each neuron is connected to every neuron in the previous layer. This type of layer is commonly used in neural networks because it allows for the transformation of the input data into more abstract representations.

Q18. Why matrix representation?

Matrix representation allows for efficient computation in neural networks by leveraging vectorized operations. Instead of iterating over each element in the input, we can perform matrix multiplication to compute the outputs for all neurons in a layer simultaneously, significantly speeding up training and inference.

Q19. What happens if there is no activation function?

Without an activation function, the network would become purely linear, and multiple layers would collapse into a single linear transformation. This would prevent the network from learning complex patterns, as it would only be able to model linear relationships between input and output.

Q20. What is the bias trick?

The bias trick is a technique used to incorporate the bias term into the weight matrix,

simplifying the model. By adding an extra input $x_0 = 1$, we can treat the bias as a weight, making it easier to implement and compute the network's output.

4.3 Backpropagation and Training

Q21. What is backpropagation?

Backpropagation is a supervised learning algorithm used for training neural networks. It involves calculating the gradient of the loss function with respect to each weight by applying the chain rule of calculus. This gradient is then used to update the weights in the direction that minimizes the loss function.

In backpropagation, the process is as follows: 1. Compute the forward pass and obtain the output. 2. Calculate the loss by comparing the predicted output with the true label. 3. Compute the gradients of the loss with respect to the weights (using the chain rule). 4. Update the weights using gradient descent.

Q22. How does backpropagation work?

Backpropagation works by propagating the error from the output layer back to the input layer, adjusting weights to reduce the error at each layer. The key idea is to compute the gradient of the loss function with respect to each weight in the network and update the weights accordingly. This is done through the following steps: 1. Calculate the output of the network for the given input. 2. Compute the error by comparing the predicted output with the actual output. 3. For each layer, compute the gradient of the error with respect to the weights and biases, then propagate this gradient backwards through the network using the chain rule. 4. Update the weights and biases to minimize the error by using an optimization algorithm like gradient descent.

Q23. What is the loss function in neural networks?

The loss function is a function that measures how well the neural network's predictions match the actual labels. For classification problems, the **cross-entropy loss** is commonly used:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

where y_i is the true label and $\hat{y}_i$ is the predicted probability. For regression problems, the **mean squared error (MSE)** loss is used:

$$L = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$$

The goal of training is to minimize the loss function by adjusting the weights using backpropagation.

Q24. What is gradient descent?

Gradient descent is an optimization algorithm used to minimize the loss function in machine learning models, including neural networks. It works by updating the model's parameters in the direction of the steepest descent of the loss function, which is determined by the negative gradient of the loss. The update rule is:

$$\theta = \theta - \eta \nabla_{\theta} L(\theta)$$

where θ represents the model parameters, η is the learning rate, and $\nabla_{\theta}L(\theta)$ is the gradient of the loss function with respect to the parameters.

Q25. What is the learning rate in gradient descent?

The learning rate is a hyperparameter that controls the step size in the gradient descent algorithm. It determines how much the model's parameters are updated with each iteration. If the learning rate is too high, the algorithm might overshoot the optimal solution, while if it is too low, the algorithm might converge too slowly or get stuck in local minima.

Q26. What is the vanishing gradient problem?

The vanishing gradient problem occurs when the gradients of the loss function become very small as they are propagated back through the network, especially in deep networks. This leads to very slow learning and prevents the model from learning effectively. The problem is most commonly seen with activation functions like the sigmoid and tanh, which squash their outputs to small values, leading to small gradients.

4.4 Deep Learning and Neural Language Models

Q27. What is a deep neural network?

A deep neural network (DNN) is a neural network with multiple hidden layers between the input and output layers. The depth of the network allows it to model more complex relationships between inputs and outputs, making it suitable for tasks like image recognition, speech processing, and natural language processing. Deep neural networks are capable of learning hierarchical representations of data, which helps them capture intricate patterns.

Q28. What are the key differences between shallow and deep networks?

- **Shallow networks** have one or two layers (input layer and output layer), and they are capable of learning only simple, linear relationships between input and output. - **Deep networks** have many hidden layers, allowing them to learn complex, non-linear relationships between input and output. These networks are capable of capturing high-level abstractions and hierarchies in data.

Q29. What is the role of hidden layers in deep neural networks?

Hidden layers in deep neural networks allow the network to learn complex features and representations of the input data. Each hidden layer applies a transformation to the input data, extracting higher-level features that are passed on to subsequent layers. The deeper the network, the more abstract and complex the learned representations become.

Q30. What are Convolutional Neural Networks (CNNs)?

Convolutional Neural Networks (CNNs) are a specialized type of neural network designed for processing grid-like data, such as images. CNNs use convolutional layers that apply filters (kernels) to the input data to extract local features, such as edges, textures, and patterns. These local features are then combined to form more complex features as they are passed through deeper layers of the network. CNNs are widely used in image classification, object detection, and other computer vision tasks.

Q31. What are Recurrent Neural Networks (RNNs)?

Recurrent Neural Networks (RNNs) are a type of neural network designed for sequential data, such as time series or text. RNNs have connections that loop back on themselves, allowing them to maintain a memory of previous inputs. This makes them well-suited for tasks like

speech recognition, language modeling, and machine translation. However, traditional RNNs struggle with long-range dependencies due to the vanishing gradient problem.

Q32. What is Long Short-Term Memory (LSTM)?

Long Short-Term Memory (LSTM) is a type of RNN designed to address the vanishing gradient problem. LSTMs have special gates (input, forget, and output gates) that regulate the flow of information, allowing the network to remember important information over longer time spans. LSTMs are commonly used in tasks like language modeling, machine translation, and speech recognition.

4.5 Neural Language Models

Q33. What is a neural language model?

A neural language model is a type of model used to predict the probability distribution of the next word in a sequence, given the previous words. These models use neural networks to learn the structure of language and generate realistic sequences of words. Neural language models are used in applications like machine translation, text generation, and speech recognition.

Q34. How do neural language models differ from traditional language models?

Traditional language models, such as **n-gram models**, estimate the probability of the next word based on a fixed context (e.g., the previous $n-1$ words). In contrast, neural language models learn continuous, dense representations of words (word embeddings) and can capture long-range dependencies between words. Neural models are more flexible and can generalize better to unseen sequences compared to traditional models.

Q35. What is Word2Vec?

Word2Vec is a popular neural network-based model for learning word embeddings. It uses shallow neural networks to map words to continuous vector representations. Word2Vec has two main architectures: - **Continuous Bag of Words (CBOW)**: Predicts a target word from its surrounding context words. - **Skip-gram**: Predicts the context words given a target word. Word2Vec embeddings capture semantic relationships between words, allowing similar words to have similar vector representations.

Q36. What is GloVe?

GloVe (Global Vectors for Word Representation) is another model for learning word embeddings. Unlike Word2Vec, which is a predictive model, GloVe is a count-based model that factorizes the word co-occurrence matrix to produce embeddings. GloVe captures both global and local word relationships and has been widely used for various NLP tasks.

Q37. What are the applications of neural language models?

Neural language models are widely used in NLP applications such as: - **Machine Translation**: Translating text from one language to another. - **Text Generation**: Generating coherent and meaningful text based on a given prompt. - **Speech Recognition**: Converting spoken language into text. - **Sentiment Analysis**: Classifying the sentiment of text as positive, negative, or neutral. - **Question Answering**: Automatically answering questions based on a text corpus.

5 Question set 1:

5.1 Question 1:

5.1.1 Part A: How can overfitting be detected in a machine learning model? What are the general techniques you can use to reduce overfitting?

Detection of Overfitting:

Overfitting occurs when a model performs well on the training data but poorly on unseen test data. This happens because the model learns the noise and irrelevant patterns in the training data rather than generalizing the underlying trends. Overfitting can be detected through various methods:

1. **Train-Test Split Performance:** If a model shows significantly better performance on the training set compared to the test set, it is likely overfitting. For example, if the accuracy on the training data is high (e.g., 98%) but the accuracy on the test data is much lower (e.g., 60%), this indicates overfitting.
2. **Cross-Validation:** Cross-validation involves splitting the data into several subsets (folds) and training the model on different combinations of these subsets. If the model shows a large variance in performance across different folds, this may indicate overfitting.
3. **Learning Curves:** Plotting learning curves of both the training and validation errors over time can help detect overfitting. If the training error keeps decreasing while the validation error starts increasing after a certain point, overfitting is likely occurring.

Techniques to Reduce Overfitting:

1. **Regularization (L1 and L2):** Regularization techniques add a penalty to the loss function to discourage the model from fitting noise. L2 regularization (Ridge) penalizes large weights by adding a term $\lambda \sum w_i^2$ to the loss function. L1 regularization (Lasso) adds a term $\lambda \sum |w_i|$, which can also lead to sparse models by setting some weights to zero.
2. **Cross-Validation:** Using cross-validation ensures that the model is validated on multiple subsets of data, helping to assess its generalizability and preventing overfitting to a single train-test split.
3. **Early Stopping:** Early stopping involves halting the training process once the validation error starts increasing, even if the training error is still decreasing. This prevents the model from learning too much noise in the training data.
4. **Dropout (for Neural Networks):** Dropout randomly disables a fraction of neurons during training, which forces the network to learn more robust features and prevents it from over-relying on specific neurons.
5. **Simplifying the Model:** Reducing the complexity of the model, such as limiting the number of features or hidden layers, can prevent overfitting. Simpler models are less likely to memorize the training data and more likely to generalize well.

5.1.2 Part B: You have built a spam detector model, and that model was tested on 100 emails. Out of these 100 emails, 80 were spams and 20 were not spams. Your model identified 90 of them as spam and 10 of them not spams. All the not spams the model identified are correct. Now, build a confusion matrix and calculate precision, recall, and F-score for the spam class.

Confusion Matrix:

Let's define the components of the confusion matrix for the spam class:

- **True Positives (TP):** The number of correctly identified spam emails.
- **False Positives (FP):** The number of non-spam emails incorrectly identified as spam.
- **True Negatives (TN):** The number of correctly identified non-spam emails.
- **False Negatives (FN):** The number of spam emails incorrectly identified as non-spam.

From the problem: - Total emails = 100 - 80 emails are spam, and 20 are non-spam.
 - The model correctly identifies all non-spam emails (10), so there are **0 False Positives** and **20 True Negatives**. - Out of the 80 spam emails, the model correctly identifies 70 as spam, leaving **10 False Negatives** (80 - 70). - The model misclassifies 10 non-spam emails as spam, so there are **10 False Positives**.

The confusion matrix looks like this:

	Predicted Spam	Predicted Not Spam
Actual Spam	70 (TP)	10 (FN)
Actual Not Spam	10 (FP)	20 (TN)

Calculating Precision, Recall, and F-Score:

1. Precision (for Spam Class):

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{70}{70 + 10} = \frac{70}{80} = 0.875$$

2. Recall (for Spam Class):

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{70}{70 + 10} = \frac{70}{80} = 0.875$$

3. F-Score (F1-Score) for Spam Class:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{0.875 \times 0.875}{0.875 + 0.875} = 0.875$$

—

5.2 Question set 2:

5.2.1 Part A: Suppose you have a collection of 3 documents and a term "AI" appears in them as follows:

- Document 1 (D1): it has 100 words and "AI" appears 3 times. - Document 2 (D2): it has 200 words and "AI" appears 5 times. - Document 3 (D3): it has 150 words and "AI" appears 0 times.

Now calculate **TF-IDF** for "AI" in all three of these documents.

Solution:

1. **Term Frequency (TF)** is the ratio of the number of times a term appears in a document to the total number of terms in the document:

$$\text{TF} = \frac{\text{Count of term in document}}{\text{Total words in document}}$$

For "AI" in each document:

$$\text{TF}(D1) = \frac{3}{100} = 0.03, \quad \text{TF}(D2) = \frac{5}{200} = 0.025, \quad \text{TF}(D3) = \frac{0}{150} = 0$$

2. **Inverse Document Frequency (IDF)** measures the importance of a term in the corpus:

$$\text{IDF}(\text{AI}) = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing the term}} \right)$$

"AI" appears in 2 documents, so:

$$\text{IDF}(\text{AI}) = \log \left(\frac{3}{2} \right) = 0.1761$$

3. **TF-IDF** for each document:

$$\begin{aligned} \text{TF-IDF}(D1) &= 0.03 \times 0.1761 = 0.00528, \\ \text{TF-IDF}(D2) &= 0.025 \times 0.1761 = 0.0044025, \\ \text{TF-IDF}(D3) &= 0 \times 0.1761 = 0 \end{aligned}$$

—

5.2.2 Part B: With appropriate examples, write short notes on (what it is, what are the challenges, how to solve them etc.):

1. Named Entity Recognition (NER):

Named Entity Recognition is the task of identifying and classifying named entities (such as names of people, organizations, locations, dates, etc.) in text. Challenges include recognizing new or ambiguous entities, disambiguating between entities with the same name, and dealing with unseen entities. NER can be solved using rule-based systems, statistical models, and more recently, deep learning techniques like Conditional Random Fields (CRFs) and LSTMs.

2. Tokenization:

Tokenization is the process of splitting text into smaller units (tokens) such as words, phrases,

or characters. Challenges in tokenization include handling punctuation, abbreviations, and multi-word expressions (e.g., "New York" or "U.S.A"). Modern tokenizers use methods like whitespace-based tokenization or machine learning-based models to handle edge cases effectively.

3. Stemming and Lemmatization:

Stemming and lemmatization are techniques used to reduce words to their base or root form. Stemming uses heuristic rules to cut off prefixes or suffixes (e.g., "running" → "run"), while lemmatization uses a dictionary or vocabulary to map words to their base form (e.g., "better" → "good"). Lemmatization is more accurate but computationally more expensive than stemming.

5.3 Question 3:

5.3.1 Part A: The Book presents three semantic properties of word embeddings: Different types of Similarity or Association, Analogy/Relational Similarity, and Historical Semantics. Write short notes on each of them with appropriate examples.

1. Similarity or Association:

Similarity refers to the degree to which two words are alike in meaning. In word embeddings, this is quantified by measuring the cosine similarity between their vectors. Words that have a similar meaning will have word vectors that are close to each other in the vector space. For example, the words "king" and "queen" are semantically similar, as they both refer to royalty. Their word vectors will be positioned close to each other in the high-dimensional vector space.

$$\text{cosine similarity}(w_1, w_2) = \frac{w_1 \cdot w_2}{\|w_1\| \|w_2\|}$$

This measure calculates how close the two word vectors are in terms of angle. If the similarity score is close to 1, the words are similar in meaning.

Example: "king" and "queen" are similar in meaning as both belong to royalty, so their embeddings will be close, yielding a high cosine similarity. Similarly, words like "cat" and "dog" are semantically similar in the context of pets or animals.

2. Analogy/Relational Similarity:

Relational similarity refers to the relationships between words that can be expressed as analogies. A classic example of this is the analogy: "man" is to "woman" as "king" is to "queen". Word embeddings can capture such relationships through simple arithmetic. For example, by performing vector arithmetic, we can generate the analogy:

$$\text{king} - \text{man} + \text{woman} = \text{queen}$$

This equation demonstrates that we can use word embeddings to solve analogies. In this case, the relationship between "king" and "queen" is similar to the relationship between "man" and "woman." This ability to solve analogies is a powerful feature of word embeddings, allowing models to understand relational semantics beyond simple word similarity.

3. Historical Semantics:

Historical semantics refers to the evolution of word meanings over time. Over the years, many words undergo semantic shifts, where their meanings change based on context, culture, or usage. Word embeddings trained on data spanning multiple time periods can capture such shifts in meaning. For instance, the word "gay" has evolved in meaning from "happy" to "homosexual" over time.

Example: Consider the word "cool," which initially referred to a temperature-related meaning (cold or chilly) but now also carries the meaning of something "fashionable" or "impressive." A well-trained word embedding model would be able to reflect this shift by placing "cool" in different regions of the vector space based on the context in which it is used. Historical semantics is especially challenging because it requires training models on large and diverse datasets that span different time periods to capture the evolution of meanings accurately.

5.3.2 Part B: You have 2 models, model A: $y_A = \sigma(2x_1 + 4x_2 - 3)$ and model B: $y_B = \sigma(3x_1 + 4x_2)$. For a data point $x = [x_1, x_2] = [0, 1]$ with label $y = 1$, which model incurs higher binary cross-entropy loss?

Solution:

To calculate the ****binary cross-entropy loss****, we use the formula:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

where: - y is the true label (in this case, $y = 1$), - $\hat{y}$ is the predicted probability for the given input x , which is computed using the sigmoid function $\sigma(z)$.

The sigmoid function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Let's evaluate both models step by step:

1. For model A:

The output of model A is:

$$y_A = \sigma(2x_1 + 4x_2 - 3)$$

Substituting $x_1 = 0$ and $x_2 = 1$:

$$y_A = \sigma(2(0) + 4(1) - 3) = \sigma(1)$$

Using the sigmoid function:

$$\sigma(1) = \frac{1}{1 + e^{-1}} \approx 0.731$$

So, the predicted probability for model A is approximately $\hat{y}_A = 0.731$.

Now, compute the binary cross-entropy loss for model A:

$$L_A = -[1 \log(0.731) + (1 - 1) \log(1 - 0.731)] = -\log(0.731) \approx 0.313$$

2. For model B:

The output of model B is:

$$y_B = \sigma(3x_1 + 4x_2)$$

Substituting $x_1 = 0$ and $x_2 = 1$:

$$y_B = \sigma(3(0) + 4(1)) = \sigma(4)$$

Using the sigmoid function:

$$\sigma(4) = \frac{1}{1 + e^{-4}} \approx 0.982$$

So, the predicted probability for model B is approximately $\hat{y}_B = 0.982$.

Now, compute the binary cross-entropy loss for model B:

$$L_B = -[1 \log(0.982) + (1 - 1) \log(1 - 0.982)] = -\log(0.982) \approx 0.018$$

Comparison:

- **Model A** incurs a loss of approximately **0.313**. - **Model B** incurs a loss of approximately **0.018**.

Therefore, **model A** incurs higher binary cross-entropy loss for the given data point.

6 Question set 2:

6.1 Question 1:

6.1.1 Part A: You are training a deep neural network, and the training accuracy is 98%, but the validation accuracy is only 70%. What does this indicate, and how can you address it?

Solution:

This situation indicates that the model is **overfitting**. The high training accuracy suggests that the model is memorizing the training data rather than learning generalizable patterns. The lower validation accuracy indicates that the model does not generalize well to unseen data. Overfitting occurs when the model becomes too complex and learns the noise or irrelevant details in the training data.

How to Address Overfitting:

1. **Regularization (L1 and L2):** Adding a penalty term to the loss function, such as L1 (Lasso) or L2 (Ridge) regularization, discourages large model weights and helps prevent overfitting.
2. **Early Stopping:** In neural networks, training can be stopped early if the validation accuracy starts to decrease, even if the training error continues to decrease.
3. **Cross-Validation:** Use k-fold cross-validation to ensure that the model is validated on multiple data splits, ensuring that it generalizes well across different subsets.

4. **Simplifying the Model:** Reduce the complexity of the model by decreasing the number of layers or neurons (in neural networks) to avoid fitting unnecessary patterns in the data.
5. **Increase Training Data:** More data can help the model generalize better and avoid overfitting. Data augmentation techniques can also be used for increasing the effective size of the dataset.

6.1.2 Part B: What is the impact of increasing the number of features in a dataset on underfitting and overfitting? Explain.

Solution:

The impact of increasing the number of features depends on the nature of the features and the model complexity.

1. **Impact on Underfitting:** If the model is too simple, increasing the number of features can help the model better capture the underlying patterns in the data, reducing underfitting. Underfitting occurs when the model is too simple to learn the relationship between the inputs and outputs, leading to poor performance on both training and test data. By adding more features, the model may have more information to work with, allowing it to fit the data better.
2. **Impact on Overfitting:** Increasing the number of features can lead to overfitting, especially if the added features are irrelevant or noisy. When a model has too many features, it may begin to fit the noise in the data, resulting in high training accuracy but poor generalization to new, unseen data. This is because the model becomes more flexible and capable of fitting complex relationships in the training data, but it may lose its ability to generalize.

6.1.3 Part C: Is overfitting always bad? Discuss one scenario where a slightly overfit model might be acceptable.

Solution:

Overfitting is generally undesirable because it leads to poor generalization. However, there are some scenarios where a slightly overfit model might be acceptable:

- **Small Dataset:** If the dataset is very small and additional features help capture nuances in the data, a slightly overfit model might be acceptable. In such cases, the model might not generalize well, but the value of capturing all available information outweighs the potential cost of overfitting.
- **High Importance on Accuracy:** In certain applications, such as medical diagnostics, even a slightly overfit model might be acceptable if it increases accuracy on the task at hand. For example, when predicting rare diseases from medical data, a slightly overfitted model might help detect critical instances more accurately.

6.2 Question 2:

6.2.1 Part A: For a dataset with 3 examples, the target values y and predicted probabilities y' are given as follows:

$$y = [1, 0, 1], \quad y' = [0.9, 0.1, 0.6]$$

Calculate the average binary cross-entropy loss for this dataset.

Solution:

The binary cross-entropy loss for each example is given by the formula:

$$L = -[y \log(y') + (1 - y) \log(1 - y')]$$

For each example:

- For the first example ($y = 1, y' = 0.9$):

$$L_1 = -[1 \log(0.9) + (1 - 1) \log(1 - 0.9)] = -\log(0.9) \approx 0.1054$$

- For the second example ($y = 0, y' = 0.1$):

$$L_2 = -[0 \log(0.1) + (1 - 0) \log(1 - 0.1)] = -\log(0.9) \approx 0.1054$$

- For the third example ($y = 1, y' = 0.6$):

$$L_3 = -[1 \log(0.6) + (1 - 1) \log(1 - 0.6)] = -\log(0.6) \approx 0.5108$$

The average binary cross-entropy loss is:

$$\text{Average Loss} = \frac{L_1 + L_2 + L_3}{3} = \frac{0.1054 + 0.1054 + 0.5108}{3} \approx 0.2405$$

Thus, the average binary cross-entropy loss for this dataset is approximately **0.2405**.

6.2.2 Part B: Describe:

a. Conditional independence and the Naive assumption of the Naive Bayes algorithm.

Solution:

In Naive Bayes, **conditional independence** refers to the assumption that the features (or attributes) are conditionally independent given the class label. This means that the presence or absence of one feature does not provide any additional information about the presence or absence of another feature once the class label is known.

The **Naive assumption** in the Naive Bayes algorithm is that, given the class label, the features are independent. This simplification allows for easy computation of the likelihood of the data. The joint probability of observing a set of features $x_1, x_2, \dots, x_n$ given a class y is assumed to be:

$$P(x_1, x_2, \dots, x_n | y) = \prod_{i=1}^n P(x_i | y)$$

This simplifies the computation, but it often leads to inaccuracies if the features are not truly independent.

b. How Naive Bayes algorithms deal with unseen features (e.g., words that were not seen during training)

Solution:

Naive Bayes algorithms handle unseen features (such as unseen words in text classification) using **Laplace smoothing** (additive smoothing). This technique prevents zero probabilities for unseen features by adding a small constant (usually 1) to the count of all features. The smoothed probability estimate is:

$$P(x_i|y) = \frac{\text{count}(x_i, y) + 1}{\text{count}(y) + V}$$

Where V is the size of the vocabulary (the total number of unique features). This way, even if a feature has never appeared in the training data, the model will still assign it a non-zero probability.

6.3 Question 3:

6.3.1 Part A: A company is analyzing customer reviews using NLP techniques. Consider the following matrix for three documents and three terms:

Document	Term: "product"	Term: "excellent"	Term: "price"
d1	3	0	1
d2	1	2	0
d3	0	1	4

Calculate TF-IDF for every term in every document.

Solution:

1. **Term Frequency (TF):** The term frequency for each term in a document is:

$$\text{TF}(w) = \frac{\text{count of term in document}}{\text{total terms in document}}$$

For each document:

- For $d1$, the total number of terms is $3 + 0 + 1 = 4$. Thus:

$$\begin{aligned}\text{TF}(\text{product}, d1) &= \frac{3}{4} = 0.75, \\ \text{TF}(\text{excellent}, d1) &= \frac{0}{4} = 0, \\ \text{TF}(\text{price}, d1) &= \frac{1}{4} = 0.25\end{aligned}$$

- For $d2$, the total number of terms is $1 + 2 + 0 = 3$. Thus:

$$\begin{aligned}\text{TF}(\text{product}, d2) &= \frac{1}{3} = 0.333, \\ \text{TF}(\text{excellent}, d2) &= \frac{2}{3} = 0.667, \\ \text{TF}(\text{price}, d2) &= \frac{0}{3} = 0\end{aligned}$$

- For $d3$, the total number of terms is $0 + 1 + 4 = 5$. Thus:

$$\begin{aligned}\text{TF}(\text{product}, d3) &= \frac{0}{5} = 0, \\ \text{TF}(\text{excellent}, d3) &= \frac{1}{5} = 0.2, \\ \text{TF}(\text{price}, d3) &= \frac{4}{5} = 0.8\end{aligned}$$

2. ****Inverse Document Frequency (IDF):**** The IDF for each term is:

$$\text{IDF}(w) = \log \left(\frac{N}{\text{Number of documents containing } w} \right)$$

Where N is the total number of documents (3 in this case). For each term: - $\text{IDF}(\text{product}) = \log \left(\frac{3}{2} \right) = 0.1761$ - $\text{IDF}(\text{excellent}) = \log \left(\frac{3}{3} \right) = 0$ - $\text{IDF}(\text{price}) = \log \left(\frac{3}{3} \right) = 0$

3. ****TF-IDF Calculation:****

For each term in each document, we calculate TF-IDF:

- For $d1$:

$$\begin{aligned}\text{TF-IDF}(\text{product}, d1) &= 0.75 \times 0.1761 = 0.1321, \\ \text{TF-IDF}(\text{excellent}, d1) &= 0 \times 0 = 0, \\ \text{TF-IDF}(\text{price}, d1) &= 0.25 \times 0 = 0\end{aligned}$$

- For $d2$:

$$\begin{aligned}\text{TF-IDF}(\text{product}, d2) &= 0.333 \times 0.1761 = 0.0587, \\ \text{TF-IDF}(\text{excellent}, d2) &= 0.667 \times 0 = 0, \\ \text{TF-IDF}(\text{price}, d2) &= 0 \times 0 = 0\end{aligned}$$

- For $d3$:

$$\begin{aligned}\text{TF-IDF}(\text{product}, d3) &= 0 \times 0.1761 = 0, \\ \text{TF-IDF}(\text{excellent}, d3) &= 0.2 \times 0 = 0, \\ \text{TF-IDF}(\text{price}, d3) &= 0.8 \times 0 = 0\end{aligned}$$

—

6.3.2 Part B: Explain how the TF-IDF values would change if the term "price" appeared in every document.

Solution:

If the term "price" appears in every document, the IDF for "price" becomes:

$$\text{IDF}(\text{price}) = \log\left(\frac{3}{3}\right) = 0$$

Since the IDF of "price" is now 0, the TF-IDF values for "price" will also be 0 in all documents, regardless of the term frequency in those documents. This is because the term becomes less informative when it appears in every document, so the model will no longer consider it useful in differentiating the documents.

7 Question Set 3:

7.1 Question 1:

7.1.1 Part A: Explain with examples:

- **Precision:**

Precision is the ratio of correctly predicted positive instances to the total predicted positive instances. It measures how many of the predicted positives are actually correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

For example, in a spam classifier, if the model predicts 100 emails as spam, but 80 of them are actually spam, the precision is:

$$\text{Precision} = \frac{80}{100} = 0.8$$

- **Micro-average F1:**

Micro-average F1 score is calculated by aggregating the contributions of all classes to compute a global precision and recall, and then calculating the F1 score. It's typically used for multi-class classification.

$$\text{Micro F1} = 2 \times \frac{\text{Micro Precision} \times \text{Micro Recall}}{\text{Micro Precision} + \text{Micro Recall}}$$

It is useful when you want to give equal weight to all data points, regardless of class imbalance.

- **Macro-average Recall:**

Macro-average recall is calculated by first computing the recall for each class individually and then taking the average.

$$\text{Macro Recall} = \frac{1}{N} \sum_{i=1}^N \text{Recall}_i$$

Macro-average recall gives equal weight to all classes, even if some classes are under-represented.

- **Statistical Significance:**

Statistical significance is a measure that determines if the results of an experiment or hypothesis test are likely to be due to something other than random chance. If a model shows a significant improvement over a baseline, this means that the results are not just a product of random variation.

7.1.2 Part B: You have two classifiers $y = \sigma(w^T x + b)$, classifier A with weight $w_A = [1, 0, 2]^T$ and bias $b_A = 1$, and classifier B with weight $w_B = [1, 2, 0]^T$ and bias $b_B = 0$. For one example $X = [1, 1, 0]^T$, predict y_A and y_B . Which classifier incurs lower cross-entropy loss if the original label of X is 1?

Solution:

The prediction of each classifier is given by the logistic sigmoid function:

$$y = \sigma(w^T X + b) = \frac{1}{1 + e^{-(w^T X + b)}}$$

For Classifier A:

$$w_A^T X + b_A = 1(1) + 0(1) + 2(0) + 1 = 2$$

$$y_A = \sigma(2) = \frac{1}{1 + e^{-2}} \approx 0.8808$$

For Classifier B:

$$w_B^T X + b_B = 1(1) + 2(1) + 0(0) + 0 = 3$$

$$y_B = \sigma(3) = \frac{1}{1 + e^{-3}} \approx 0.9526$$

The cross-entropy loss is:

$$L = -[y \log(y') + (1 - y) \log(1 - y')]$$

For X 's true label $y = 1$:

For Classifier A:

$$L_A = -[1 \log(0.8808) + 0 \log(1 - 0.8808)] \approx -\log(0.8808) \approx 0.1278$$

For Classifier B:

$$L_B = -[1 \log(0.9526) + 0 \log(1 - 0.9526)] \approx -\log(0.9526) \approx 0.0488$$

Thus, **Classifier B incurs a lower cross-entropy loss**.

—

7.2 Question 2:

7.2.1 Part A: Suppose you have a collection of 3 documents and a term "AI" appears in them as follows:

Document	Term: "product"	Term: "excellent"	Term: "price"
d1	3	0	1
d2	1	2	0
d3	0	1	4

Now calculate **TF-IDF** for "AI" in all three of these documents.

Solution:

1. ****Term Frequency (TF):**** The term frequency for each term in a document is given by:

$$\text{TF} = \frac{\text{Count of term in document}}{\text{Total terms in document}}$$

For each document: - For $d1$, the total number of terms is $3 + 0 + 1 = 4$. Thus: - $\text{TF}(\text{product}, d1) = \frac{3}{4} = 0.75$ - $\text{TF}(\text{excellent}, d1) = \frac{0}{4} = 0$ - $\text{TF}(\text{price}, d1) = \frac{1}{4} = 0.25$

- For $d2$, the total number of terms is $1 + 2 + 0 = 3$. Thus: - $\text{TF}(\text{product}, d2) = \frac{1}{3} = 0.333$ - $\text{TF}(\text{excellent}, d2) = \frac{2}{3} = 0.667$ - $\text{TF}(\text{price}, d2) = \frac{0}{3} = 0$

- For $d3$, the total number of terms is $0 + 1 + 4 = 5$. Thus: - $\text{TF}(\text{product}, d3) = \frac{0}{5} = 0$ - $\text{TF}(\text{excellent}, d3) = \frac{1}{5} = 0.2$ - $\text{TF}(\text{price}, d3) = \frac{4}{5} = 0.8$

2. ****Inverse Document Frequency (IDF):**** The IDF for each term is:

$$\text{IDF}(w) = \log \left(\frac{N}{\text{Number of documents containing } w} \right)$$

Where N is the total number of documents (3 in this case). For each term: - $\text{IDF}(\text{product}) = \log \left(\frac{3}{2} \right) = 0.1761$ - $\text{IDF}(\text{excellent}) = \log \left(\frac{3}{3} \right) = 0$ - $\text{IDF}(\text{price}) = \log \left(\frac{3}{3} \right) = 0$

3. ****TF-IDF Calculation:****

For each term in each document, we calculate TF-IDF:

• For $d1$:

$$\text{TF-IDF}(\text{product}, d1) = 0.75 \times 0.1761 = 0.1321,$$

$$\text{TF-IDF}(\text{excellent}, d1) = 0 \times 0 = 0,$$

$$\text{TF-IDF}(\text{price}, d1) = 0.25 \times 0 = 0$$

• For $d2$:

$$\text{TF-IDF}(\text{product}, d2) = 0.333 \times 0.1761 = 0.0587,$$

$$\text{TF-IDF}(\text{excellent}, d2) = 0.667 \times 0 = 0,$$

$$\text{TF-IDF}(\text{price}, d2) = 0 \times 0 = 0$$

• For $d3$:

$$\text{TF-IDF}(\text{product}, d3) = 0 \times 0.1761 = 0,$$

$$\text{TF-IDF}(\text{excellent}, d3) = 0.2 \times 0 = 0,$$

$$\text{TF-IDF}(\text{price}, d3) = 0.8 \times 0 = 0$$

7.2.2 Part B: Explain how the TF-IDF values would change if the term "price" appeared in every document.**Solution:**

If the term "price" appears in every document, the IDF for "price" becomes:

$$\text{IDF}(\text{price}) = \log\left(\frac{3}{3}\right) = 0$$

Since the IDF of "price" is now 0, the TF-IDF values for "price" will also be 0 in all documents, regardless of the term frequency in those documents. This is because the term becomes less informative when it appears in every document, so the model will no longer consider it useful in differentiating the documents.

7.3 Question 3:**7.3.1 Part A: Gender bias in word embeddings means some words are closer to male words and other words are closer to female words. For example, a poorly trained word vector may associate 'doctor' to 'man' and 'nurse' to 'woman'. If I tell you your embedding has this issue, how will you test it? How can you solve it? Explain.****Solution:**

To test gender bias in word embeddings, you can examine the cosine similarity between gender-specific words and other words. For example: - Compute the cosine similarity between "doctor" and gendered terms like "man" and "woman." - Check whether gendered terms (e.g., "nurse", "doctor") cluster with "man" or "woman" in the embedding space.

One solution to mitigate gender bias is to use **gender-neutral embeddings**, where the bias is minimized by either debiasing the word embeddings or training models on gender-balanced data. One such method is to use **hard debiasing** techniques, where the gender direction is identified and removed from the embedding space.

7.3.2 Part B: Explain the training and testing process of the Naive Bayes algorithm for sentiment classification. That is, you are given N sentences where some are positive sentiment sentences and some are negative, and you have to train a Naive Bayes model that will be able to classify a sentence. Then, you got a new sentence which you do not know whether it is positive and negative, and you need to figure out. How are you going to do it?**Solution:**

The Naive Bayes algorithm for sentiment classification works in the following way:

- **Training:**

- Calculate the prior probabilities for each class (positive and negative sentiment). For example, $P(\text{positive})$ and $P(\text{negative})$.

- For each class, calculate the likelihood of each word given the class:

$$P(\text{word}_i \mid \text{positive})$$

- Use the **Laplace smoothing** technique to handle words that may not appear in the training data.
- **Testing:** For a new sentence, calculate the posterior probability for both the positive and negative classes using Bayes' theorem:

$$P(\text{positive}|\text{sentence}) \propto P(\text{positive}) \prod_{i=1}^n P(\text{word}_i|\text{positive})$$

Similarly for the negative class. Classify the sentence by choosing the class with the higher posterior probability.